

Lexique référence

Lexique TyrannoScript vers Godot 4.4



GESTION DE LA CAMÉRA

TyrannoScript

```
[camera time="0" zoom="1.000001" wait="false" layer="base" ease_type="linear"]  
[reset_camera time="0" wait="true" layer="base"]
```

Équivalent Godot 4.4

- **Nœud** : `Camera2D` OU `Camera3D`
- **Scripts** :
 - `camera.zoom = Vector2(zoom_value, zoom_value)`
 - `create_tween().tween_property(camera, "zoom", target_zoom, duration)`
 - `camera.position = Vector2.ZERO` (reset)



GESTION AUDIO

TyrannoScript

```
[playse volume="70" time="1000" buf="0" storage="SoundEffects_Menu/file.ogg"]  
[playbgm time="1000" loop="true" storage="Music/track.ogg"]  
[fadeoutbgm time="2000"]  
[wse] # Wait Sound End
```

Équivalent Godot 4.4

- **Nœuds** : `AudioStreamPlayer` , `AudioStreamPlayer2D`
- **Scripts** :

```
gdscript
# Sound Effect
func play_se(file_path: String, volume: float = 0.0):
    audio_se.stream = load(file_path)
    audio_se.volume_db = linear_to_db(volume/100.0)
    audio_se.play()

# Background Music
func play_bgm(file_path: String, loop: bool = true, fade_time: float = 1.0):
    var new_stream = load(file_path)
    new_stream.loop = loop
    var tween = create_tween()
    tween.tween_method(fade_bgm, 0.0, 1.0, fade_time)

# Fade Out BGM
func fadeout_bgm(duration: float):
    var tween = create_tween()
    tween.tween_property(audio_bgm, "volume_db", -80.0, duration)
    tween.tween_callback(audio_bgm.stop)
```



GESTION DES IMAGES ET ARRIÈRE-PLANS

TyrannoScript

```
[tb_image_show time="1000" storage="path/image.png" layer="1"]
[tb_image_hide time="1000"]
[bg storage="Backgrounds/bg.png" method="crossfade" time="1000"]
[image storage="file.png" layer="0" method="crossfade"]
```

```
[freeimage layer="0"]  
[tb_cg id="Space"]
```

Équivalent Godot 4.4

- **Nœuds** : `TextureRect`, `Sprite2D`, `CanvasLayer`
- **Scripts** :

```
gdscript  
# Afficher image avec fondu  
func show_image(texture_path: String, layer: int, duration: float = 1.0):  
    var sprite = Sprite2D.new()  
    sprite.texture = load(texture_path)  
    get_layer(layer).add_child(sprite)  
    sprite.modulate.a = 0.0  
    var tween = create_tween()  
    tween.tween_property(sprite, "modulate:a", 1.0, duration)  
  
# Changer arrière-plan avec transition  
func change_bg(texture_path: String, transition_type: String = "crossfade",  
duration: float = 1.0):  
    var new_bg = TextureRect.new()  
    new_bg.texture = load(texture_path)  
    new_bg.expand_mode = TextureRect.EXPAND_FIT_WIDTH_PROPORTION  
    AL  
  
    match transition_type:  
        "crossfade":  
            crossfade_transition(current_bg, new_bg, duration)  
        "fade":  
            fade_transition(new_bg, duration)
```

SYSTÈME DE DIALOGUE

TyrannoScript

```
[position layer="message1" width="1280" height="720" top="0" left="0"]
[tb_show_message_window]
[current layer="message1"]
[layopt layer="message0" visible="false"]
[[] [r] [p] # Line break, clear text, paragraph end
```

Équivalent Godot 4.4

- **Nœuds** : `RichTextLabel` , `Panel` , `CanvasLayer`
- **Scripts** :

```
gdscript
# Configuration de la fenêtre de message
func setup_message_window(layer_name: String, size: Vector2, position: Vector2):
    var message_layer = CanvasLayer.new()
    message_layer.name = layer_name
    var panel = Panel.new()
    panel.size = size
    panel.position = position

    var rich_text = RichTextLabel.new()
    rich_text.fit_content = true
    rich_text.bbcode_enabled = true
    panel.add_child(rich_text)
    message_layer.add_child(panel)

# Gestion du texte avec balises - CORRECTION
func process_text_tags(text: String) -> String:
    text = text.replace("[l]", "[wait_click]")# Wait for click (pause)
    text = text.replace("[r]", "\n")# Return/Line break
    text = text.replace("[p]", "[wait_click][clear_text]")# Page break (pause +
```

```
clear)
return text
```

GESTION DES PERSONNAGES

TyrannoScript

```
[chara_show name="Klaus" time="1000" storage="chara/klaus.png"]
[chara_mod name="Klaus" storage="chara/klaus_smile.png" time="200"]
[chara_hide name="Klaus" time="500"]
[chara_hide_all time="500"]
```

Équivalent Godot 4.4

- **Nœuds** : `Sprite2D` , `AnimationPlayer`
- **Scripts** :

```
gdscript
# Dictionnaire des personnages
var characters = {}

# Afficher personnage
func show_character(name: String, sprite_path: String, duration: float = 1.0):
    var character = Sprite2D.new()
    character.texture = load(sprite_path)
    character.name = name
    characters[name] = character
    add_child(character)

# Animation d'entrée
character.modulate.a = 0.0
var tween = create_tween()
```

```

tween.tween_property(character, "modulate:a", 1.0, duration)

# Modifier expression du personnage
func modify_character(name: String, new_sprite: String, duration: float = 0.2):
    if name in characters:
        var tween = create_tween()
        tween.tween_property(characters[name], "modulate:a", 0.0, duration/2)
        tween.tween_callback(func(): characters[name].texture = load(new_sprite))
        tween.tween_property(characters[name], "modulate:a", 1.0, duration/2)

```

FORMATAGE DU TEXTE

TyrannoScript

```

[tb_pTEXT_show x="434" y="313" size="100" color="0xf2f2f2" time="1000"]
#<font face="earthorbiter"><font color="#94b8b8">Speaker Name</font>

```

Équivalent Godot 4.4

- **Nœuds** : `RichTextLabel`, `Label`
- **Scripts** :

```

gdscript
# Texte stylisé
func show_styled_text(text: String, pos: Vector2, size: int, color: Color, font: String = ""):
    var label = RichTextLabel.new()
    label.position = pos

```

```

label.add_theme_font_size_override("normal_font_size", size)
label.add_theme_color_override("default_color", color)

if font != "":
    var font_resource = load("res://fonts/" + font + ".ttf")
    label.add_theme_font_override("normal_font", font_resource)

label.text = text
add_child(label)

# Système de noms de personnages avec couleurs
var speaker_colors = {
    "Klaus": Color("#cc66ff"),
    "Euphie": Color("#ff8080"),
    "Computer": Color("#94b8b8"),
    "Pilot": Color("#ffb366")
}

func format_dialogue(speaker: String, text: String) → String:
    var color_hex = speaker_colors.get(speaker, Color.WHITE).to_html()
    return "[color=" + color_hex + "]" + speaker + "[/color]: " + text

```

TEMPORISATION ET FLUX

TyrannoScript

```

[wait time="2500"]
[fadeoutse time="2000" layer="3"]

```

Équivalent Godot 4.4

- **Timer, Tween**
- **Scripts :**

```
gdscript
# Attendre
func wait_time(duration_ms: int):
    await get_tree().create_timer(duration_ms / 1000.0).timeout

# Fondu sonore
func fadeout_sound(audio_player: AudioStreamPlayer, duration: float):
    var tween = create_tween()
    tween.tween_property(audio_player, "volume_db", -80.0, duration)
    tween.tween_callback(audio_player.stop)
```

TRANSITIONS ET EFFETS

TyrannoScript

```
method="crossfade" time="1000"
effect="fadeOutLeft"
```

Équivalent Godot 4.4

- **AnimationPlayer, Tween**
- **Scripts :**

```
gdscript
enum TransitionType {
    CROSSFADE,
    FADE,
    SLIDE_LEFT,
    SLIDE_RIGHT
}
```

```

func apply_transition(node: Node, type: TransitionType, duration: float = 1.0):
    var tween = create_tween()

    match type:
        TransitionType.CROSSFADE:
            tween.tween_property(node, "modulate:a", 0.0, duration/2)
            tween.tween_property(node, "modulate:a", 1.0, duration/2)

        TransitionType.FADE:
            tween.tween_property(node, "modulate:a", 1.0, duration)

        TransitionType.SLIDE_LEFT:
            node.position.x = -get_viewport().size.x
            tween.tween_property(node, "position:x", 0, duration)

```

ARCHITECTURE RECOMMANDÉE POUR GODOT

Structure de scène principale

```

Main (Node2D)
├── BackgroundLayer (CanvasLayer) [layer = -10]
│   └── Background (TextureRect)
├── CharacterLayer (CanvasLayer) [layer = 0]
│   └── Characters (Node2D)
├── UILayer (CanvasLayer) [layer = 10]
│   ├── DialogueBox (Panel)
│   └── TextDisplay (RichTextLabel)
└── AudioManager (Node)
    ├── BGMPlayer (AudioStreamPlayer)
    ├── SEPlayer (AudioStreamPlayer)
    └── VoicePlayer (AudioStreamPlayer)

```

Script Manager principal

```

gdscript
extends Node

class_name DialogueManager

signal dialogue_finished
signal choice_made(choice_id: int)

var current_script: Array = []
var current_line: int = 0
var is_playing: bool = false

func load_script(script_path: String):
# Charger et parser le script TyrannoScript
    current_script = parse_tyranoscript(script_path)

func execute_next_command():
    if current_line >= current_script.size():
        dialogue_finished.emit()
        return

    var command = current_script[current_line]
    await process_command(command)
    current_line += 1

    if is_playing:
        execute_next_command()

```



SYSTÈME DE DONNÉES RECOMMANDÉ

Format JSON pour les scripts convertis

```

json

```

```

{
  "script_name": "scene1",
  "commands": [
    {
      "type": "camera",
      "params": {"time": 0, "zoom": 1.0, "wait": false}
    },
    {
      "type": "playse",
      "params": {"volume": 70, "file": "SoundEffects_Menu/chapter.ogg"}
    },
    {
      "type": "dialogue",
      "params": {
        "speaker": "Klaus",
        "text": "Welcome to SS04.",
        "speaker_color": "#cc66ff"
      }
    }
  ]
}

```

Ce lexique vous donne une base solide pour recréer toutes les fonctionnalités de TyrannoScript dans Godot 4.4. Chaque section contient les équivalents directs et les implémentations recommandées.